Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification

Functional Pearl

AUTHOR NAME, Institution, Country

Pre- and post-conditions are the standard vocabulary for modular program specification, yet they are silent about obligations that span multiple calls. In real Haskell code such obligations are ubiquitous: a successful TLS handshake must eventually be followed by `bye` before the context is discarded; a taken `MVar` must eventually be put back on *every* code path, including those interrupted by asynchronous exceptions; a `beginTx` must eventually reach `commit` or `rollback`; a submitted job must complete with probability at least p. No single pre/post pair can express these properties, because the balancing operation lies in an unknown future context. Existing remedies — `bracket`, `ResourceT`, `Acquire` — enforce *cleanup* but cannot express *protocol* obligations or quantitative commitments.

We present *Pledge*, a Haskell library that fills this gap by augmenting the classical contract model with *future conditions*: annotations that propagate obligations forward through a computation until they are fully discharged. Future conditions are carried as a data-dependent field `future` :: $a \to \mathit{eff}$ in the `Pledge` monad alongside pre- and post-conditions. The dependence on the return value is essential: it allows an obligation to name the specific resource handle that an operation produces — requiring, for instance, that the *particular* context returned by `tlsHandshake` must eventually have `bye` called on it. When operations are sequenced via monadic bind, the future condition of the first computation is partially discharged by the postcondition of its continuation through a *subtraction* operator; the residual is then conjoined with the continuation's own future condition. The user annotates individual operations only; propagation and discharge are handled automatically by the monad.

The design is parameterised over a `Composable` algebra requiring concatenation ($\diamond$), conjunction ($\wedge$), and subtraction ($\setminus$), together with identity elements `empty` and `universe`. We present four instances: trace protocols via extended regular expressions (RE, with Brzozowski-derivative subtraction and a direct embedding of LTL_f); trace protocols enriched with Presburger arithmetic bounds (GRE, discharged by an SMT solver); quantitative obligations via semiring-weighted regular expressions (WRE, capturing probabilistic and min-cost scenarios); and heap-ownership obligations via separation logic (SL, with magic wand as subtraction). All four instances share a single monadic bind formula, demonstrating that *Pledge* subsumes trace-based, arithmetic, quantitative, and heap-ownership obligations within one algebraic interface.

CCS Concepts: • **Software and its engineering → Functional languages**; *Data types and structures*; *Software verification and validation.*

Additional Key Words and Phrases: temporal specification, monadic effects, regular expressions, complement, Brzozowski derivatives, Presburger arithmetic, semiring weights, separation logic, resource lifecycle, Haskell

ACM Reference Format:
Author Name. 2026. Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification: Functional Pearl. *Proc. ACM Program. Lang.* 1, HASKELL (May 2026), 25 pages.

Author's Contact Information: Author Name, Institution, City, Country, author@institution.edu.

ACM 2475-1421/2026/5-ART
https://doi.org/

1 Introduction

Pre- and post-conditions are the workhorse of modular program specification. A pre-condition constrains the state in which an operation may be invoked; a post-condition describes what holds immediately after it returns. Yet a great many correctness properties are not local to a single call boundary: they *span* a sequence of operations, with arbitrary computation in between.

The problem in real Haskell code. Consider three scenarios from the Haskell ecosystem.

TLS lifecycle. A successful `handshake` must eventually be followed by `bye` before the context is discarded. A post-condition on `handshake` records success, but cannot enforce that `bye` will be called, because the discharge point lies in an unknown future context. `bracket` sidesteps this by lexically scoping the context, but cannot express richer message-exchange obligations or handle contexts passed across function boundaries.

MVar discipline. `takeMVar` removes a value, leaving the variable empty; the caller must restore it with `putMVar` on every exit path including async-exception paths. A post-condition records the empty state but cannot require that a future `putMVar` on the *same* variable will occur.

Database savepoints. `transactionSave` opens a fresh transaction that must itself be committed or rolled back; pre/post-conditions on the call cannot carry this obligation forward.

In each case the obstacle is identical: the postcondition cannot name what comes next because the continuation is unknown at specification time.

Existing remedies and their limits. `bracket` lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. `ResourceT` [**?**] and `Acquire`/`Managed` thread cleanup actions through every bind but model only the cleanup half of an obligation, discharging it uniformly at scope exit rather than by application-specific operations. Linear Haskell [**?**] enforces "consume exactly once" but cannot express branching discharge obligations or quantitative commitments. Parameterised and indexed monads [**?**] encode protocol states at the type level but require `RebindableSyntax` or `QualifiedDo`, produce opaque type errors, and are confined to a single domain per design.

Our approach. We propose the `Pledge` monad as a lightweight, compositional mechanism for specifying such obligations, requiring no language extensions beyond standard Haskell type classes. A value of type `Pledge eff a` represents a computation that returns a value of type a and carries three annotations: a precondition `pre` :: *eff*, a postcondition `post` :: *eff*, and a data-dependent future condition `future` :: $a \to$ *eff* that expresses what the remaining execution must satisfy *as a function of the return value.* The dependence on the return value is essential: it allows an obligation to refer to the specific resource handle that an operation produces. For example, `tlsHandshake` can declare that *whichever* `Context` it returns must eventually have `bye` called on it: not a context fixed at specification time, but the one actually negotiated at runtime. When two computations are sequenced via monadic bind, the future condition of the first is *partially discharged* by the postcondition of the continuation, using the `subtraction` operator of the `Composable` algebra; the residual is then conjoined with the continuation's own future condition. The user annotates individual operations only; propagation is handled automatically by the monad.

The `Composable` typeclass abstracts the specification algebra into five operations: concatenation ($\diamond$), conjunction ($\wedge$), subtraction ($\backslash$), and their respective identities `empty` and `universe`. It admits four independent instances:

- *(Trace protocol,* RE*)* Extended regular expressions over events, with complement as a first-class constructor. Subtraction is implemented via Brzozowski derivatives; LTL_f operators embed directly as RE terms without automaton construction. This instance handles `takeMVar`/`putMVar` discipline, TLS handshake/bye pairing, transaction lifecycle, and lock protocols.

- *(Arithmetic bounds,* GRE*)* Each RE is paired with a Presburger arithmetic predicate over program variables, evaluated by an SMT solver (Z3 via SBV) at membership-check time. A single annotation simultaneously enforces a protocol ordering *and* a numeric bound, catching, for example, counter overflow before any execution takes place.
- *(Quantitative obligation,* WRE*)* Boolean membership is generalised to a semiring weight. The probability semiring captures obligations of the form "this job completes with probability at least p"; the tropical semiring gives minimum-cost scheduling obligations. A weight of `szero` in the residual signals an unfulfilled quantitative commitment.
- *(Heap ownership,* SL*)* Separation-logic heap predicates, with separating conjunction as sequential composition and magic wand as subtraction. Future conditions describe what heap state must hold when the computation completes; a leaked `Cell` in the residual post names the unreleased ownership precisely.

All four instances share a single monadic bind formula, demonstrating that trace-based, arithmetic, quantitative, and heap-ownership obligations are not merely analogous; they are structurally identical at the level of the `Composable` algebra.

Contributions.

(1) We define the `Pledge` monad (§3), parameterised over a `Composable` algebra, with `pre` :: *eff*, `post` :: *eff*, and `future` :: $a \to$ *eff* fields. The `future` field is *data-dependent*: it is a function of the return value, so obligations can refer to the resource handle an operation produces. We verify that the monad laws hold for the (*ret*, *post*, *future*) triple (§9).
(2) We instantiate the algebra to extended regular expressions over events with complement, using Brzozowski derivatives to implement subtraction (§2.1), and show that LTL_f operators embed directly as RE terms (§2.3).
(3) We extend the RE instance to a guarded form (GRE, §5), pairing each RE with a Presburger arithmetic predicate over program variables, evaluated by an SMT solver (Z3 via SBV) at membership-check time.
(4) We provide a weighted RE instance (WRE, §6), parameterised over a semiring, that generalises Boolean membership to a quantitative weight, recovering probabilistic and min-cost future conditions as special cases.
(5) We provide a fourth, independent instantiation to separation-logic heap predicates (§7), demonstrating that the `Composable` interface extends beyond trace-based specifications.
(6) We demonstrate the approach on ten use cases across all four instances (§8), showing that resource leaks, ordering violations, arithmetic overflows, probabilistic obligations, and heap-ownership errors all surface as non-trivial residual annotations.

2 Background

2.1 Extended Regular Expressions over Events

Events are the atomic units of observation; each carries a name and a list of typed arguments.

```
data Term  = Str String | Num Int | List [Term]

data Event = Atom String [Term]   -- concrete event, e.g. Atom "send" [Num 1]
           | Wildcard             -- pattern matching any single event

data RE = Bot | Epsilon | Single Event
        | Seq RE RE | Or RE RE | And RE RE | Star RE | Not RE
```

The specification language is a regular expression type that adds complement as a first-class constructor:

$$r \;::=\; \varnothing \mid \varepsilon \mid e \mid r_1 \cdot r_2 \mid r_1 \vee r_2 \mid r_1 \wedge r_2 \mid r^* \mid \neg r$$

Here e is a `Single Event` that uses `Wildcard` as a pattern for any event, $\wedge$ is intersection (derived: $r_1 \wedge r_2 \triangleq \neg(\neg r_1 \vee \neg r_2)$), and $\neg r$ is the complement with respect to Σ^*.

Five derived forms are used throughout:

$$\begin{aligned}
\texttt{anything} &\triangleq \neg\varnothing && \text{(the universal language } \Sigma^*\text{)}\\
\texttt{finally}(e) &\triangleq \texttt{anything} \cdot e \cdot \texttt{anything} && \text{(}e\text{ must occur eventually)}\\
\texttt{globally}(e) &\triangleq e^* && \text{(}e\text{ at every step)}\\
\texttt{never}(e) &\triangleq \neg\texttt{finally}(e) && \text{(}e\text{ must never occur)}\\
\texttt{noUntil}(e, g) &\triangleq \neg\big((\Sigma \setminus \{g\})^* \cdot e \cdot \texttt{anything}\big) && \text{(}e\text{ must not occur before }g\text{)}
\end{aligned}$$

`noUntil`(e, g) is nullable (the empty continuation satisfies it) and its derivative with respect to g is Σ^*, meaning once g occurs the restriction on e is lifted. This is the key combinator for *conditional* safety: e is forbidden until g re-enables it.

2.2 Algebraic Complement and Brzozowski Derivatives

The Brzozowski derivative [Brzozowski 1964] $\partial_a(r)$ computes the language of continuations after reading event a. The key law for complement is:

$$\partial_a(\neg r) \;=\; \neg\big(\partial_a(r)\big) \tag{1}$$

The nullability function $\nu(r)$ (= `True` iff $\varepsilon \in L(r)$) satisfies $\nu(\neg r) = \neg\,\nu(r)$. Together, equations (1) and the De Morgan laws

$$\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2 \qquad \neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$$

allow complement to be propagated and simplified *purely algebraically*, without building a DFA. The normaliser additionally applies $\neg\neg r = r$, $\neg\varnothing = \neg\varnothing$, and $\neg\neg\varnothing = \varnothing$.

In Haskell:

```
nullable :: RE → Bool
nullable (Not r)     = not (nullable r)      -- nu(Not r) = not(nu(r))
nullable (Star _)    = True
nullable (Seq r1 r2) = nullable r1 && nullable r2
nullable (Or  r1 r2) = nullable r1 || nullable r2
nullable (And r1 r2) = nullable r1 && nullable r2
nullable Epsilon     = True
nullable _           = False

derivative :: Event → RE → RE
derivative e (Not r)      = Not (derivative e r)   -- d_a(Not r) = Not(d_a(r))
derivative e (Single p)   = if matchEvent e p then Epsilon else Bot
derivative e (Seq r1 r2)
    | nullable r1 = Or (Seq (derivative e r1) r2) (derivative e r2)
    | otherwise   = Seq (derivative e r1) r2
```

```
derivative e (Or  r1 r2)  = Or  (derivative e r1) (derivative e r2)
derivative e (And r1 r2)  = And (derivative e r1) (derivative e r2)
derivative e (Star r)     = Seq (derivative e r) (Star r)
derivative _ _            = Bot
```

The `firstWith` function returns the set of events (drawn from a supplied finite alphabet Σ_0) that can begin a word in $L(r)$, driving the subtraction algorithm. The complement case requires particular care. Naively writing `first (Not r) = first r` is *unsound*: $L(\neg r)$ can begin with events *not* in $\mathit{first}(r)$, so the naive rule explores the wrong direction. The correct characterisation is:

$$e \in \mathit{first}(\neg r, \Sigma_0) \iff e \in \Sigma_0 \ \wedge\ \lceil \partial_e(r) \rceil \neq \neg\varnothing \tag{2}$$

That is, e can open a word in $L(\neg r)$ iff after consuming e, the residual $\partial_e(r)$ does *not* cover the whole future; there exists some continuation that stays outside $L(r)$. The alphabet Σ_0 is collected from an RE via `atoms`, which recursively gathers all concrete (`Atom`) events, excluding `Wildcard`.

```
atoms :: RE → [Event]
atoms (Single Wildcard) = []
atoms (Single e)        = [e]
atoms (Seq r1 r2)       = atoms r1 `union` atoms r2
atoms (Or  r1 r2)       = atoms r1 `union` atoms r2
atoms (And r1 r2)       = atoms r1 `union` atoms r2
atoms (Star r)          = atoms r
atoms (Not r)           = atoms r
atoms _                 = []

firstWith :: [Event] → RE → [Event]
firstWith _    Bot              = []
firstWith _    Epsilon          = []
firstWith _    (Single e)       = [e]
firstWith alph (Seq r1 r2)
    | nullable r1               = firstWith alph r1 `union` firstWith alph r2
    | otherwise                 = firstWith alph r1
firstWith alph (Or  r1 r2)     = firstWith alph r1 `union` firstWith alph r2
firstWith alph (And r1 r2)     = [e | e <- firstWith alph r1,
                                      e `elem` firstWith alph r2]
firstWith alph (Star r)        = firstWith alph r
firstWith alph (Not r)         = [e | e <- alph,
                                      not (isTotal (normalize (derivative e r)))]
  where isTotal (Not Bot) = True; isTotal _ = False
```

The function `first r = firstWith (atoms r) r` is a convenient wrapper for standalone use; inside `subtraction` both operands contribute to the alphabet (see §4).

Illustrative cases. Table 1 gives representative evaluations of `firstWith` with `Not`. The first four rows confirm that complement does not exclude any alphabet event when the derivative is non-total. The double-negation row shows correct restoration of the original first set: $\partial_b(\neg a) = \Sigma^*$ is total, so b is filtered out. In $\neg(a \cdot \Sigma^*)$, $\partial_a(a \cdot \Sigma^*)$ normalises to Σ^*, excluding a; only b survives. Similarly, $\neg(a \cdot \Sigma^* \vee b \cdot \Sigma^*)$ excludes both a and b, leaving only c, while the De Morgan row verifies that $\neg(\neg a \wedge \neg b) = a \vee b$ yields $\{a, b\}$.

Table 1. Selected `firstWith` evaluations with **`Not`**. Σ_0 is the explicitly supplied alphabet. Rows marked † produce a *strict* subset of Σ_0.

RE r	Σ_0	`firstWith` Σ_0 $(\neg r)$	Rationale
a	$\{a, b\}$	$\{a, b\}$	$\partial_a(a) = \varepsilon$, $\partial_b(a) = \varnothing$; neither Σ^*
$a \cdot b$	$\{a, b, c\}$	$\{a, b, c\}$	all derivatives non-Σ^*
$a \vee b$	$\{a, b, c\}$	$\{a, b, c\}$	$\partial_c(a \vee b) = \varnothing$, still non-$\Sigma^*$
a^*	$\{a, b\}$	$\{a, b\}$	$\partial_a(a^*) = a^*$, non-Σ^*; b starts $[b]$
$\neg a$ (double neg.)	$\{a, b\}$	$[a]$	$\partial_b(\neg a) = \Sigma^* \Rightarrow b$ excluded†
$a \cdot \Sigma^*$	$\{a, b\}$	$[b]$	$\partial_a(a \cdot \Sigma^*) = \Sigma^* \Rightarrow a$ excluded†
$a \cdot \Sigma^* \vee b \cdot \Sigma^*$	$\{a, b, c\}$	$[c]$	both a, b excluded; only c avoids†
$\neg a \wedge \neg b$	$\{a, b, c\}$	$\{a, b\}$	De Morgan: $= \neg(a \vee b)$; ∂_c is Σ^*†

2.3 Embedding $\mathrm{LTL_f}$

Because complement is a first-class constructor, the standard $\mathrm{LTL_f}$ operators translate directly into RE, with no automaton construction needed. Table 2 gives the full mapping.

Table 2. $\mathrm{LTL_f}$ to RE translation. $\Sigma^1 \triangleq$ `Single Wildcard`, $\neg\varnothing \triangleq \neg\varnothing$.

LTL formula	RE $[\![\cdot]\!]$	
$\top$	$\neg\varnothing$	
$\bot$	$\varnothing$	
e	$\{e\}$	
$\neg\phi$	$\neg[\![\phi]\!]$	
$\phi \wedge \psi$	$[\![\phi]\!] \cap [\![\psi]\!]$	
$\phi \vee \psi$	$[\![\phi]\!] \cup [\![\psi]\!]$	
$X\,\phi$	$\Sigma^1 \cdot [\![\phi]\!]$	
$F\,\phi$	$\neg\varnothing \cdot [\![\phi]\!]$	
$G\,\phi$	$\neg(\neg\varnothing \cdot \neg[\![\phi]\!])$	
$\phi\,U\,\psi$	$\mathit{step}(\phi)^* \cdot [\![\psi]\!]$	(ϕ propositional)

The complement operator does the heavy lifting: $G\,\phi \triangleq \neg F \neg\phi$ unfolds to $\neg(\neg\varnothing \cdot \neg[\![\phi]\!])$ using two applications of $\neg$; $F\,\phi$ and $X\,\phi$ need only concatenation. The U case uses $\mathit{step}(\phi)$, the length-1 RE for a single event satisfying ϕ, which is defined only when ϕ is propositional.

3 The `Pledge` Monad

3.1 The `Composable` Class

We abstract the specification algebra via a type class:

```
class Composable a where
  concatenation :: a → a → a   -- (⋄), unit: empty
  conjunction   :: a → a → a   -- (∧), unit: universe
  empty         :: a           -- identity for ⋄  (varepsilon)
  universe      :: a           -- identity for ∧  (neg-emptyset)
  subtraction   :: a → a → a   -- (\\)
```

The five operations form exactly the interface required by the bind formula. `concatenation` sequences effects; `conjunction` combines simultaneous constraints; `subtraction` computes the residual obligation (the Brzozowski quotient for the RE instance). `empty` and `universe` are the respective identity elements and appear in `pure`.

3.2 The `Pledge` Type

The `future` field is *data-dependent*: it is a function of the return value `a`, allowing obligations to refer to the specific resource handle that an operation produces. The helper `evalFuture` evaluates this function at the computation's own return value.

```
data Pledge eff a = Pledge
  { ret    :: a
  , pre    :: eff        -- what must have occurred before
  , post   :: eff        -- what this computation produces
  , future :: a → eff    -- obligation indexed by the return value
  }

evalFuture :: Pledge eff a → eff
evalFuture e = future e (ret e)
```

3.3 Monad Instances

`pure` introduces a value with no effect and no obligation:

```
pure x = Pledge
  { ret = x, pre = universe, post = empty, future = \_ → universe }
```

The bind operator sequences two computations; write $e_2 = f(ret(e))$.

```
e >>= f = let fe = f (ret e) in Pledge
  { ret    = ret fe
  , pre    = pre e ∧ (post e \\ pre fe)
  , post   = post e ⋄ post fe
  , future = \_ → (post fe \\ future e (ret e)) ∧ future fe (ret fe)
  }
```

Future condition. Each `future` function is applied to the corresponding `ret` before the obligation is propagated:

$$future(e \gg\!= f) \;=\; \big(post(e_2) \setminus future(e)(ret(e))\big) \;\wedge\; future(e_2)(ret(e_2)) \tag{3}$$

This removes from e's future obligation (evaluated at e's return value) those traces that e_2's postcondition satisfies, then intersects the residual with e_2's own future obligation (evaluated at e_2's return value). When the result normalises to $\neg\varnothing$, all obligations are discharged.

Precondition.

$$pre(e \gg\!= f) \;=\; pre(e) \;\wedge\; \big(post(e) \setminus pre(e_2)\big) \tag{4}$$

The term $post(e) \setminus pre(e_2)$ is the *residual precondition*: the part of e_2's precondition not already discharged by e's postcondition. The combination uses $\wedge$ (intersection): both $pre(e)$ and the residual must hold *simultaneously* in the same input history.

When $post(e)$ does not cover $pre(e_2)$ at all, the residual is $\varnothing$, flagging a precondition violation. When $post(e)$ exactly satisfies $pre(e_2)$, the residual is ε and the combined precondition becomes

$pre(e) \wedge \varepsilon$. If $pre(e) = \neg\varnothing$ (nullable), this reduces to ε; if $pre(e) = \{a\}$ (non-nullable), the intersection $\{a\} \wedge \varepsilon = \varnothing$ flags the contradiction immediately. This behaviour is intentional: incompatible constraints collapse to the empty intersection rather than silently discarding one side.

3.4 Intuition via an Example

`takeMVar` returns the handle of the taken `MVar` as its return value; the future condition is data-dependent, referring to whatever handle was actually returned. `putMVar` asserts that `putMVar(v)` must not recur until `takeMVar(v)` is called again, preventing double-put.

```
type MVar = Int

takeMVar :: MVar → Pledge RE MVar
takeMVar v = Pledge
  { ret = v, pre = universe
  , post   = Single (Atom "takeMVar" [Num v])
  , future = \r → finally (Atom "putMVar" [Num r]) }

putMVar :: MVar → Pledge RE ()
putMVar v = Pledge
  { ret = ()
  , pre    = Single (Atom "takeMVar" [Num v])
  , post   = Single (Atom "putMVar" [Num v])
  , future = \_ → noUntil (Atom "putMVar" [Num v]) (Atom "takeMVar" [Num v]) }
```

Good program: `v <- takeMVar 1; putMVar v`. After `takeMVar 1` ($ret = 1$):

$$post = \texttt{takeMVar}(1), \quad future = \texttt{finally}(\texttt{putMVar}(1)), \quad pre = \neg\varnothing$$

Binding `putMVar 1` (with $pre = \texttt{takeMVar}(1)$, $post = \texttt{putMVar}(1)$):

Precondition: $post(\texttt{takeMVar}(1)) \setminus pre(\texttt{putMVar}(1)) = \texttt{takeMVar}(1) \setminus \texttt{takeMVar}(1) = \varepsilon$. So $pre = \neg\varnothing \wedge \varepsilon = \varepsilon$. ✓

Future: $\texttt{putMVar}(1) \setminus \texttt{finally}(\texttt{putMVar}(1)) = \neg\varnothing$. Intersect with $future(\texttt{putMVar}(1)) = \texttt{noUntil}(\texttt{putMVar}(1$ result $\texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1))$. ✓(nullable: the program may end here; no double-put obligation is pending.)

Bad: `putMVar 1` alone. `pure ()` has $post = \varepsilon$. $\texttt{takeMVar}(1) \setminus \varepsilon = \varnothing$. So $pre = \varnothing$. ×

Bad: `missingPut` (`takeMVar 1 ≫ takeMVar 2 ≫ putMVar 1`). The future of `takeMVar 2` is $\texttt{finally}(\texttt{putMVar}(2))$ which is not discharged by `putMVar 1`, so the residual future still contains $\texttt{finally}(\texttt{putMVar}(2))$, naming the blocked `MVar` precisely.

Bad: `doublePutImmediate` (`v <- takeMVar 1; putMVar v; putMVar v`). After the first `putMVar 1`, the future is $\texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1))$. The second `putMVar 1` posts $\texttt{putMVar}(1)$; consuming $\texttt{putMVar}(1)$ in $\texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1))$ yields $\varnothing$, so the combined future is $\varnothing$. ×

4 The RE Instance

The `Composable RE` instance maps the algebra operations onto `RE` constructors and implements subtraction via derivatives:

```
instance Composable RE where
  concatenation = Seq
  conjunction   = And
  empty         = Epsilon
  universe      = anything      -- = Not Bot
```

```
  subtraction Epsilon   r2 = r2
  subtraction (Not Bot) _  = Epsilon   -- Sigma* \\ r = eps (trivially satisfied)
  subtraction r1 r2 =
    let alph   = atoms r1 `union` atoms r2   -- combined alphabet
        evts   = firstWith alph r1
        step e = subtraction (normalize (derivative e r1))
                             (normalize (derivative e r2))
    in foldr Or Bot (map step evts)

normalize :: RE → RE   -- RE-specific; not part of Composable
normalize r = {- ... algebraic simplification ... -}
```

The second clause handles the universal case: $\neg\varnothing \setminus r = \varepsilon$ for any r. The general clause collects the alphabet from *both* operands before calling `firstWith`, ensuring events appearing only in r_2 are still considered under complement via Equation (2).

`normalize` is defined outside the instance and applies the following RE-specific laws, which exploit the complement constructor:

Law	Applied to
$\varepsilon \cdot r = r, r \cdot \varepsilon = r$	`Seq`
$\varnothing \vee r = r, r \vee \neg\varnothing = \neg\varnothing$	`Or`
$\varnothing \wedge r = \varnothing, r \wedge \neg\varnothing = r$	`And`
$\neg\neg r = r$	`Not`
$\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2$	`Not`
$\neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$	`Not`
$\neg\varnothing = \neg\varnothing, \neg\neg\varnothing = \varnothing$	`Not`
$\varnothing^* = \varepsilon, \varepsilon^* = \varepsilon$	`Star`

5 The GuardedRE Instance

The RE instance reasons about event traces but is entirely silent about arithmetic state. The GRE instance closes this gap: a guarded regular expression pairs a trace RE with a Presburger arithmetic predicate over a concrete heap, so that a single annotation can simultaneously enforce a protocol ordering *and* an arithmetic bound.

5.1 The GuardedRE Type

```
data GuardedRE = GuardedRE PPred RE
    deriving (Eq)
```

A state (heap, trace) satisfies `GuardedRE p r` iff

$$\text{heap} \models p \quad \wedge \quad \text{trace} \in L(r).$$

The two sides are independent: p is a static constraint on the heap, while r advances through Brzozowski derivatives as events arrive.

Two smart constructors lift a pure RE or a pure Presburger predicate:

```
fromRE    :: RE    → GuardedRE
fromRE r   = GuardedRE PTrue r        -- no heap constraint

fromPPred :: PPred → GuardedRE
```

```
fromPPred p = GuardedRE p universe    -- accept any trace
```

5.2 The `Composable GuardedRE` Instance

```
instance Composable GuardedRE where
    concatenation (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (Seq r1 r2))
    conjunction (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (And r1 r2))
    empty    = GuardedRE PTrue Epsilon
    universe = GuardedRE PTrue (Not Bot)   -- = anything
    subtraction (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2))
                  (normalize (reSubtraction r1 r2))
```

Both `concatenation` and `subtraction` conjoin both heap predicates: the residual must respect the constraints from both operands. The RE component uses the same `reSubtraction` and `normalize` as the plain RE instance.

5.3 Presburger Normalisation

Composing multiple `GuardedRE` values accumulates a tree of `PAnd` nodes. `normalizePPred` flattens this tree, removes duplicates, substitutes equalities (e.g. $h[a] = k \Rightarrow$ replace `ValAt` a by k throughout), evaluates literal comparisons, and replaces $\neg$**true** with the canonical **false** constructor `PFalse`:

```
data PPred
    = PTrue | PFalse      -- canonical true/false
    | PLt PExpr PExpr | ...
    | PNot PPred | PAnd PPred PPred

normalizePPred :: PPred → PPred
normalizePPred p = rebuildAnd (substEqs eqs atoms)
  where
    atoms = flattenAnd (normStep p)
    eqs   = [ (a, k) | PEq (ValAt a) (Lit k) <- atoms ]
```

The introduction of `PFalse` prevents the solver from being invoked with expressions such as ($\neg$**true**) $\wedge \cdots$ that are syntactically unsatisfiable; these normalise immediately to `PFalse` without any solver call.

5.4 Membership Checking

Checking whether a concrete (*heap*, *trace*) satisfies a `GuardedRE` involves two steps: fold `deriveGuarded` over the trace (advancing only the RE side), then call `nullableGuarded` which checks RE nullability *and* invokes the SBV/Z3 backend on the instantiated Presburger predicate.

```
deriveGuarded :: Event → GuardedRE → GuardedRE
deriveGuarded e (GuardedRE p r) = GuardedRE p (normalize (derivative e r))

nullableGuarded :: Map Addr Int → GuardedRE → IO Bool
nullableGuarded heap (GuardedRE p r)
    | not (nullable r) = return False
    | otherwise = do
```

```
        result <- checkPPred (instantiate heap p)
        return $ case result of { Satisfied _ → True; _ → False }

checkGuarded :: Map Addr Int → [Event] → GuardedRE → IO Bool
checkGuarded heap trace g =
    nullableGuarded heap (foldl (flip deriveGuarded) g trace)
```

5.5 Example: Bounded Counter

A bounded counter stored at heap address a with maximum value M must follow the protocol $\texttt{init} \cdot (\texttt{inc} \mid \texttt{dec})^* \cdot \texttt{snapshot}$. Increment is additionally guarded by $h[a] < M$ (no overflow); decrement by $h[a] > 0$ (no underflow).

```
initCounter :: Addr → Pledge GuardedRE ()
initCounter a = Pledge
  { ret = ()
  , pre  = fromRE universe
  , post = GuardedRE (PEq (ValAt a) (Lit 0))
                     (Single (Atom "init" [Num a]))
  , future = \_ → fromRE universe }

increment :: Addr → Int → Pledge GuardedRE ()
increment a maxVal = Pledge
  { ret = ()
  , pre  = GuardedRE (PLt (ValAt a) (Lit maxVal)) universe
  , post = fromRE (Single (Atom "inc" [Num a]))
  , future = \_ → fromPPred (PGe (ValAt a) (Lit 0)) }
```

The `pre` of `increment` carries $h[a] < M$ (pure Presburger) conjoined with the universal RE. After composing eleven increments against a bound of 10, `normalizePPred` derives the contradiction $h[a] < 10 \wedge \neg(h[a] < 10) \equiv \mathbf{false}$, flagging the overflow before any execution takes place.

Bad programs and residuals.

Bad program	Residual `pre` (Presburger)	Residual `pre` (RE)
`overflow` (11 incs, max=10)	**false**	$\neg\varnothing$
`underflow` (dec at zero)	**false**	$\neg\varnothing$
`noInit` (skip init)	$\neg\varnothing$	$\varnothing$

6 The WeightedRE Instance

The RE and GRE instances are Boolean: a future condition is either satisfied or violated. The WRE instance generalises membership to a *weight* drawn from a semiring, enabling probabilistic and quantitative obligations to be expressed within the same `Composable` framework.

6.1 The Semiring Class

```
class (Eq w, Show w) ⇒ Semiring w where
    szero :: w          -- additive identity  (blocked path)
    sone  :: w          -- multiplicative identity
    sadd  :: w → w → w
    smul  :: w → w → w
```

Setting w = `Bool` (with `sadd` = $(\vee)$, `smul` = $(\wedge)$) recovers the Boolean RE instance. Two further instances capture quantitative scenarios:

Instance	`szero`	`sadd`	`smul`
`Prob` (probability)	0	$\min(p_1 + p_2,\ 1)$	$p_1 \times p_2$
`Tropical` (min-cost)	$+\infty$	min	$+$

In the probability semiring, `sadd` is bounded addition (sum of probabilities capped at 1) and `smul` is multiplication. In the tropical semiring, `sadd` is minimum (taking the cheaper alternative) and `smul` is addition (summing costs along a path).

6.2 The `WRE` Type and Operators

```
data WRE w
    = WBot               -- weight szero: no accepting path
    | WEps w             -- weight w: accept the empty trace
    | WSingle w Event    -- weight w: accept exactly this event
    | WSeq (WRE w) (WRE w)
    | WAdd (WRE w) (WRE w)
    | WAnd (WRE w) (WRE w)
    | WStar (WRE w)
```

The generalised nullable function accumulates the weight of all accepting paths through the empty trace:

```
wNullable :: Semiring w ⇒ WRE w → w
wNullable (WEps w)      = w
wNullable (WSeq r1 r2)  = smul (wNullable r1) (wNullable r2)
wNullable (WAdd r1 r2)  = sadd (wNullable r1) (wNullable r2)
wNullable (WAnd r1 r2)  = smul (wNullable r1) (wNullable r2)
wNullable (WStar r)     = sone
wNullable _             = szero
```

`wDerivative` and `wSubtraction` follow the same structure as their Boolean counterparts, with semiring operations substituted for Boolean ones. The `Composable (WRE w)` instance (enabled by `FlexibleInstances`) maps `concatenation` to `WSeq`, `conjunction` to `WAnd`, and `subtraction` to `wSubtraction`.

6.3 Derived Forms

```
wTop     :: Semiring w ⇒ WRE w
wTop      = WStar (WSingle sone Wildcard)   -- sone-weighted anything

wFinally :: Semiring w ⇒ w → Event → WRE w
wFinally weight e = WSeq wTop (WSingle weight e)

wGlobally :: Semiring w ⇒ w → Event → WRE w
wGlobally weight e = WStar (WSingle weight e)
```

6.4 Example: Probabilistic Memory (Prob Semiring)

In a probabilistic setting, each `free(a)` has an associated probability of being called. `malloc` carries a future condition asserting that `free(a)` must be called with probability at least p:

```
malloc :: Addr → Double → Pledge (WRE Prob) Addr
malloc a p = Pledge
  { ret = a, pre = wTop
  , post   = WSingle sone (Atom "malloc" [Num a])
  , future = \r → wFinally (Prob p) (Atom "free" [Num r]) }
```

The residual weight of `wNullable` at the end of a program measures how much of the obligation has been satisfied: a weight of `Prob 1.0` indicates that the obligation is fully discharged; a weight strictly less than 1 quantifies the probability shortfall.

6.5 Example: Min-Cost Scheduling (Tropical Semiring)

Under the tropical semiring, `WSingle (Tropical c) e` assigns cost c to emitting event e; `wNullable` then returns the minimum total cost of reaching an accepting state:

```
submitJob :: Int → Double → Pledge (WRE Tropical) ()
submitJob jobId cost = Pledge
  { ret = ()
  , pre  = wTop
  , post = WSingle (Tropical cost) (Atom "submit" [Num jobId])
  , future = \_ →
      wFinally (Tropical 0) (Atom "complete" [Num jobId]) }
```

Composing several `submitJob` calls yields a future whose `wNullable` is the minimum total cost of completing all submitted jobs. A program that omits a `complete` event for some job i retains `wFinally (Tropical 0) (complete i)` in the residual, and the overall weight is $+\infty$ (i.e. `szero`), flagging the missing completion.

7 The SL Instance

The `Composable` typeclass is not inherently tied to trace-based specifications. We demonstrate its generality with a fourth instance over *separation-logic heap predicates*, where future conditions describe what heap state must hold upon completion of the computation.

7.1 Structural Parallel with RE

Table 3 shows the role each SL construct plays in the `Composable` interface.

Table 3. Structural parallel between the four `Composable` instances.

Operation	RE	GRE	WRE_w	SL
`concatenation`	$r_1 \cdot r_2$	$(p_1 \wedge p_2,\ r_1 \cdot r_2)$	`WSeq`	$P * Q$
`conjunction`	$r_1 \wedge r_2$	$(p_1 \wedge p_2,\ r_1 \wedge r_2)$	`WAnd`	$P \wedge Q$
`empty`	ε	$(\mathbf{true}, \varepsilon)$	`WEps sone`	**emp**
`universe`	$\neg\varnothing$	$(\mathbf{true}, \neg\varnothing)$	`wTop`	$\top$
`subtraction`	Brzozowski quotient	quotient (RE) + $\wedge$ (PPred)	`wSubtraction`	$P \mathrel{-\!\!*} Q$

The bind formula is unchanged: $\mathit{pre}(e \gg\!= f) = \mathit{pre}(e) \wedge (\mathit{post}(e) \mathrel{-\!\!*} \mathit{pre}(e_2))$ and $\mathit{future}(e \gg\!= f) = (\mathit{post}(e_2) \mathrel{-\!\!*} \mathit{future}(e)) \wedge \mathit{future}(e_2)$. Here $\mathrel{-\!\!*}$ is the magic wand (separating implication): $P \mathrel{-\!\!*} Q$ holds of heap h iff for every heap h' disjoint from h satisfying P, the combined heap $h \cup h'$ satisfies Q.

7.2 The SL Predicate Type

```
data SL
    = Emp              -- empty heap         (identity for SepStar)
    | Top              -- any heap           (identity for Conj)
    | Pure PPred       -- pure Presburger constraint (heap-independent)
    | Cell Addr Val    -- singleton: address a holds value v
    | SepStar SL SL    -- P * Q  separating conjunction
    | Conj SL SL       -- P ∧ Q ordinary conjunction
    | Wand SL SL       -- P -* Q magic wand (residual)
    deriving (Eq, Show)
```

7.3 Separating vs. Ordinary Conjunction

`SepStar` P Q holds of heap h iff h splits into disjoint parts $h_1 \uplus h_2$ with $P(h_1)$ and $Q(h_2)$; it is the standard separating conjunction of ownership. `Conj` P Q holds of h iff the *same* heap satisfies both P and Q. Without pure constraints, `Conj` on purely spatial predicates is either trivial ($\top \wedge P = P$) or unsatisfiable (`Cell` l_1 $v_1 \wedge$ `Cell` l_2 v_2 = **false** for $l_1 \neq l_2$). The constructor becomes useful when one side is a pure constraint $\lceil \phi \rceil \wedge (\ell \mapsto v)$, combining a heap-independent condition with spatial ownership.

7.4 Presburger Arithmetic for Pure Constraints

Pure constraints are modelled by a small Presburger arithmetic language over heap values. Expressions are linear combinations of heap dereferences and literals; predicates are comparisons and propositional connectives.

```
data PExpr
    = Lit Int          -- integer literal
    | ValAt Addr       -- h[a]: value at address a
    | Add PExpr PExpr  -- e1 + e2
    | Mul Int PExpr    -- k * e  (scalar only, keeps linearity)

data PPred
    = PTrue
    | PLt PExpr PExpr  | PLe PExpr PExpr  | PEq PExpr PExpr
    | PGt PExpr PExpr  | PGe PExpr PExpr
    | PNot PPred       | PAnd PPred PPred
```

Derived connectives (`pFalse`, `pOr`) are smart constructors built via De Morgan laws. `Pure p` holds of any heap for which `evalPPred h p` is true; it enforces no ownership of cells. The smart constructor `pureSL` realises the standard SL box notation: `pureSL PTrue = Emp` and `pureSL p = Conj (Pure p) Emp` for non-trivial p.

7.5 The `Composable SL` Instance

```
instance Composable SL where
    concatenation = SepStar
    conjunction   = Conj
    empty         = Emp
    universe      = Top
```

```
subtraction Emp q = q           -- emp -* Q = Q
subtraction Top _ = Emp         -- Top trivially covers any pre
subtraction p   q = Wand p q    -- general: symbolic wand
```

The two base cases mirror those of the RE instance: $\varepsilon \setminus r = r$ becomes $\mathbf{emp} \mathrel{-\!\!*} Q = Q$ (providing no heap leaves the obligation unchanged), and $\neg\varnothing \setminus r = \varepsilon$ becomes $\top \mathrel{-\!\!*} Q = \mathbf{emp}$ (a $\top$ postcondition covers any precondition, leaving no residual). The general case records the wand symbolically for later evaluation or normalisation.

7.6 Normalisation

`normalizeSL` applies the standard SL algebraic identities:

Law	Constructor
$\mathbf{emp} * Q = Q$, $P * \mathbf{emp} = P$	`SepStar`
$\top \wedge Q = Q$, $P \wedge \top = P$	`Conj`
$P \wedge P = P$ (idempotent)	`Conj`
$\mathbf{emp} \mathrel{-\!\!*} Q = Q$	`Wand`
$P \mathrel{-\!\!*} \top = \top$	`Wand`

All rules apply recursively bottom-up.

8 Examples

We demonstrate `Pledge` across ten use cases spanning all four instances. Each example defines primitive operations annotated with `pre`, `post`, and `future`, composes them in the `Pledge` monad, and inspects the normalised annotations of the result. A `future` of $\neg\varnothing$ (or `wNullable = sone` for WRE) and a `pre` of $\neg\varnothing$ together certify temporal correctness.

8.1 TLS Handshake Lifecycle

The `tls` library obligation is representative of a broad class of Haskell protocol obligations: a successful `handshake` must always be paired with a subsequent `bye`. We model the `Context` as an integer identifier.

```
type Context = Int

tlsHandshake :: Context → Pledge RE Context
tlsHandshake ctx = Pledge
  { ret    = ctx
  , pre    = universe
  , post   = Single (Atom "handshake" [Num ctx])
  , future = \c → finally (Atom "bye" [Num c]) }

tlsBye :: Context → Pledge RE ()
tlsBye ctx = Pledge
  { ret    = ()
  , pre    = Single (Atom "handshake" [Num ctx])
  , post   = Single (Atom "bye" [Num ctx])
  , future = \_ → universe }

tlsSend :: Context → Pledge RE ()
tlsSend ctx = Pledge
```

```
    { ret    = ()
    , pre    = Single (Atom "handshake" [Num ctx])
    , post   = Single (Atom "send" [Num ctx])
    , future = \_ → universe }

-- Good: handshake followed by data exchange then bye
goodSession = do
  ctx <- tlsHandshake 1
  tlsSend ctx
  tlsBye ctx

-- Bad future: bye omitted; residual = finally(bye(1))
missingBye = do
  ctx <- tlsHandshake 1
  tlsSend ctx

-- Bad future: two contexts opened, only one closed
partialClose = do
  ctx1 <- tlsHandshake 1
  ctx2 <- tlsHandshake 2
  tlsBye ctx1
```

`missingBye` retains `finally(bye(1))` as its residual future, naming the unclosed context precisely. `partialClose` retains `finally(bye(2))`: the future condition is data-dependent, so the residual identifies context 2 specifically, not a generic "some context was not closed."

8.2 MVar Lifecycle

`takeMVar` returns the taken handle; its data-dependent future condition uses the lambda `r` so the obligation tracks the *returned* handle. `putMVar` carries `noUntil(putMVar(`v`), takeMVar(`v`))` as its future, forbidding a second `putMVar(v)` until `takeMVar(v)` resets the guard.

```
type MVar = Int

takeMVar :: MVar → Pledge RE MVar
takeMVar v = Pledge
  { ret = v, pre = universe
  , post   = Single (Atom "takeMVar" [Num v])
  , future = \r → finally (Atom "putMVar" [Num r]) }

putMVar :: MVar → Pledge RE ()
putMVar v = Pledge
  { ret = ()
  , pre    = Single (Atom "takeMVar" [Num v])
  , post   = Single (Atom "putMVar" [Num v])
  , future = \_ → noUntil (Atom "putMVar" [Num v]) (Atom "takeMVar" [Num v]) }

-- Good: data-dependent  -  obligation discharged on the returned handle
takeAndPut n = do { v <- takeMVar n; putMVar v }

-- Good: re-acquire same MVar after releasing  -  noUntil is reset by takeMVar
```

```
takePutTakePut = do { v <- takeMVar 1; putMVar v; v <- takeMVar 1; putMVar v }

-- Bad future: putMVar(2) obligation remains (MVar 2 never returned)
missingPut = do { v1 <- takeMVar 1; putMVar v1; _ <- takeMVar 2; return () }

-- Bad pre: takeMVar(1) precondition not met
putWithoutTake = putMVar 1

-- Bad future: double-put immediately collapses future to Bot
doublePutImmediate = do { v <- takeMVar 1; putMVar v; putMVar v }

-- Bad future: double-put with intervening work, still collapses to Bot
doublePutWithWork = do
  { v1 <- takeMVar 1; v2 <- takeMVar 2; putMVar v1; putMVar v2; putMVar v1 }
```

The three examples below return non-unit values, showing that future conditions are tracked regardless of the return type.

```
-- Good: ret = MVar; future = finally(putMVar(1))  -  obligation visible in residual
takeReturnHandle :: Pledge RE MVar
takeReturnHandle = takeMVar 1

-- Bad future: ret = (MVar,MVar); future = finally(putMVar(1)) ∧ finally(putMVar(2))
takeTwoReturnPair :: Pledge RE (MVar, MVar)
takeTwoReturnPair = do { v1 <- takeMVar 1; v2 <- takeMVar 2; return (v1, v2) }

-- Good: ret = MVar; future = noUntil(putMVar(1),takeMVar(1))  -  no pending finally
takePutReturnHandle :: Pledge RE MVar
takePutReturnHandle = do { v <- takeMVar 1; putMVar v; return v }
```

8.3 Mutex / Lock Lifecycle

```
release :: Int → Pledge RE ()
release m = Pledge
  { ret = ()
  , pre    = Single (Atom "acquire" [Num m])
  , post   = Single (Atom "release" [Num m])
  , future = universe }

-- Bad future: lock 1 not released
lockLeak = do { acquire 1; acquire 2; release 2 }

-- Bad pre: release without acquire
releaseWithoutAcquire = release 1
```

8.4 Database Transactions

`beginTx` carries a *disjunctive* future condition requiring that the transaction is eventually either committed or rolled back; `commit` and `rollback` each discharge this obligation:

```
beginTx :: Pledge RE ()
beginTx = Pledge
  { ret = (), pre = universe
  , post   = Single (Atom "beginTx" [])
  , future = Or (finally (Atom "commit" []))
                (finally (Atom "rollback" [])) }

commit :: Pledge RE ()
commit = Pledge
  { ret = (), pre = Single (Atom "beginTx" [])
  , post   = Single (Atom "commit" [])
  , future = universe }

rollback :: Pledge RE ()
rollback = Pledge
  { ret = (), pre = Single (Atom "beginTx" [])
  , post   = Single (Atom "rollback" [])
  , future = universe }

-- Bad future: transaction never closed
openTx = do { beginTx }

-- Good: transaction properly committed
committedTx = do { beginTx; commit }

-- Good: transaction properly rolled back
abortedTx = do { beginTx; rollback }
```

`openTx` retains the full disjunction $\texttt{finally(commit)} \vee \texttt{finally(rollback)}$ as its residual future. `committedTx` and `abortedTx` fully discharge the obligation, yielding $\neg\varnothing$.

RE summary. Table 4 summarises the three RE domains.

Table 4. RE instance: bad programs and residual annotations.

Domain	Bad program	Residual `future`	Residual `pre`
Memory	`missingFree`	$\texttt{finally(free(2))}$	$\neg\varnothing$
Memory	`freeWithoutMalloc`	$\neg\varnothing$	$\varnothing$
Memory	`doubleFreeImmediate`	$\varnothing$	$\neg\varnothing$
Memory	`leakAndDoubleFree`	$\varnothing$	$\neg\varnothing$
Memory	`allocReturnAddr`	$\texttt{finally(free(1))}$	$\neg\varnothing$
Memory	`allocTwoReturnPair`	$\texttt{finally(free(1))} \wedge \texttt{finally(free(2))}$	$\neg\varnothing$
Mutex	`acquire 1` leaked	$\texttt{finally(release(1))}$	$\neg\varnothing$
Mutex	release without acquire	$\neg\varnothing$	$\varnothing$
Transaction	no commit/rollback	$\texttt{finally(commit)} \vee \texttt{finally(rollback)}$	$\neg\varnothing$

8.5 Bounded Counter (GRE Instance)

The bounded counter (§5) demonstrates that both the Presburger and RE components are necessary. The RE alone would accept overflow sequences; the Presburger predicate alone would accept arbitrarily ordered events.

```
-- Good: init, two increments, one decrement, snapshot
normalUse :: Pledge GuardedRE ()
normalUse = do
  initCounter 0; increment 0 10; increment 0 10
  decrement 0;   snapshot 0

-- Bad: 11 increments against max 10
overflow :: Pledge GuardedRE ()
overflow = do
  initCounter 0
  sequence_ (replicate 11 (increment 0 10))
  snapshot 0
```

For `overflow`, the combined `pre` after composing all eleven increments normalises to $[\mathbf{false}] \wedge \neg\varnothing$: the Presburger side collapses to `PFalse` while the RE side remains the universal language, pinpointing the arithmetic violation.

GRE *summary.*

Bad program	Residual `pre` (GRE)	Cause
`overflow`	$[\mathbf{false}]\ \neg\varnothing$	$h[0] < 10$ violated
`underflow`	$[\mathbf{false}]\ \neg\varnothing$	$h[0] > 0$ violated
`noInit`	$[\mathbf{true}]\ \varnothing$	init event missing

8.6 Task Scheduling (WRE Instance)

The task-scheduling example shows that the tropical semiring naturally accumulates minimum costs. Each `submitJob` operation carries both a submission cost and a future condition requiring that every submitted job eventually completes.

```
-- Good: submit two jobs, complete both
goodSchedule :: Pledge (WRE Tropical) ()
goodSchedule = do
  submitJob 1 2.0; submitJob 2 3.0
  completeJob 1;   completeJob 2

-- Bad: job 2 never completed
missingComplete :: Pledge (WRE Tropical) ()
missingComplete = do
  submitJob 1 2.0; submitJob 2 3.0
  completeJob 1
```

For `goodSchedule`, `wNullable (evalFuture ...)` returns `Tropical 0`, indicating all obligations are discharged at zero residual cost. For `missingComplete`, the residual future still contains `wFinally (Tropical 0) (complete 2)`, and the overall weight is $+\infty$, flagging job 2 as unfinished.

The probability-semiring memory example follows analogously: a `malloc a p` with $p = 0.9$ leaves a residual weight of 0.9 if `free a` is never called, dropping to 0 when it is.

8.7 Heap Memory (SL Instance)

The heap-memory example mirrors the RE memory example but uses heap ownership rather than event traces.

```
alloc :: Addr → Val → Pledge SL ()
alloc a v = Pledge
  { ret = (), pre = Top, post = Cell a v, future = Top }

free :: Addr → Val → Pledge SL ()
free a v = Pledge
  { ret = (), pre = Cell a v, post = Emp, future = Top }

writeCell :: Addr → Val → Val → Pledge SL ()
writeCell a old new = Pledge
  { ret = (), pre = Cell a old, post = Cell a new, future = Top }
```

Good: `alloc 0 1` ≫ `alloc 1 2`. The combined `post` is `SepStar (Cell 0 1) (Cell 1 2)`: two disjoint ownerships composed by separating conjunction, correctly reflecting that the two cells reside at distinct addresses.
Bad: leak (`alloc 0 1` ≫ `alloc 1 2` ≫ `free 0 1`). After freeing only address 0, the `post` retains `Cell 1 2`, naming the unreleased ownership.
Bad: read without ownership (`readCell 0 42`). The `pre` of the bare read is `Cell 0 42`, which is never discharged, flagging the missing ownership.

8.8 Bank Account with Presburger Guards (SL Instance)

The bank account example demonstrates `Pure` Presburger constraints inside `Conj`.

```
withdraw :: Addr → Val → Val → Pledge SL ()
withdraw a old amount = Pledge
  { ret = ()
  , pre  = Conj (Pure (PGe (ValAt a) (Lit amount))) (Cell a old)
  , post = Cell a (old - amount)
  , future = Top }

closeAccount :: Addr → Val → Pledge SL ()
closeAccount a bal = Pledge
  { ret = ()
  , pre  = Conj (Pure (PEq (ValAt a) (Lit 0))) (Cell a bal)
  , post = Emp, future = Top }
```

The precondition of `withdraw` is a `Conj` of a pure Presburger guard ($h[a] \geq$ `amount`) and a spatial ownership claim (`Cell` *a* `old`). Without the `Pure` constructor, `Conj` of two purely spatial predicates would be either trivial or unsatisfiable (see §7); the combination of `Pure` and `Conj` is what makes value-dependent preconditions expressible.
Bad: overdraft. After `deposit 0 0 100`, the account holds 100. `withdraw 0 100 150` carries *pre* = `Conj (Pure` ($h[0] \geq 150$)) `(Cell 0 100)`. Since $100 < 150$, the pure guard is not satisfiable, and the residual `pre` of the composed program retains this unsatisfiable constraint.

8.9 Linked List (SL Instance)

A singly-linked-list node occupies two consecutive cells: address a holds the payload value and address a+1 holds the next pointer.

```
allocNode :: Addr → Val → Val → Pledge SL ()
allocNode a val next = Pledge
  { ret = (), pre = Top
  , post = SepStar (Cell a val) (Cell (a+1) next)
  , future = Top }
```

Two-node list (`allocNode 0 10 2 ≫ allocNode 2 20 (-1)`). The combined `post` is a four-cell `SepStar`:

$$(\texttt{Cell 0 10} * \texttt{Cell 1 2}) * (\texttt{Cell 2 20} * \texttt{Cell 3 } -1)$$

This is the canonical representation of a two-node list in separation logic: all four cells are disjointly owned, and the structure of `SepStar` mirrors the structure of the list.

SL summary. Table 5 summarises the three SL domains. The key difference from the RE instance is that residual annotations are heap predicates rather than traces: a leaked cell appears in `post`, an unsatisfiable ownership constraint in `pre`.

Table 5. SL instance: bad programs and residual annotations.

Domain	Bad program	Residual `post`	Residual `pre`
Heap memory	`Cell 1 2` not freed	`Cell 1 2` (leaked)	$\top$
Heap memory	read without ownership	**emp**	`Cell 0 42`
Bank account	insufficient balance	`Cell 0` -50	unsatisfiable guard
Bank account	close with non-zero balance	**emp**	unsatisfied $h[0] = 0$
Linked list	read without ownership	**emp**	`Cell 0 99`

9 Monad Laws

Because `future` has type `a → eff`, the monad laws are stated in terms of `evalFuture`, defined as `future e (ret e)`: the future condition evaluated at the computation's own return value. We write $\widehat{future}(e)$ for $evalFuture(e)$ for brevity. The laws hold for the triple $(ret, post, \widehat{future})$ under the assumption that `Composable` forms a monoid under $\diamond$ with unit `empty`, and that `universe` is the unit for `(∧\)`. The `pre` field satisfies the Hoare-rule property (Equation 4) rather than strict monad-law equality; we discuss this in the final paragraph of this section.

Left identity. `pure x ≫= f = f x`.
ret: $ret(f\,x)$. ✓ $\quad post$: $\varepsilon <> post(f\,x) = post(f\,x)$. ✓
For the future, let $fe = f(ret(\texttt{pure}\ x)) = f\,x$. $\texttt{future}(\texttt{pure}\ x) = \lambda\_.\ \neg\varnothing$, so $\widehat{future}(\texttt{pure}\ x) = \neg\varnothing$.

$$\begin{aligned}\widehat{future}(\texttt{pure}\ x \gg\!= f) &= \big(post(fe) \setminus \widehat{future}(\texttt{pure}\ x)\big) \wedge \widehat{future}(fe) \\ &= \big(post(f\,x) \setminus \neg\varnothing\big) \wedge \widehat{future}(f\,x) \;=\; \varepsilon \wedge \widehat{future}(f\,x) \;=\; \widehat{future}(f\,x).\ \checkmark\end{aligned}$$

The identity $r \setminus \neg\varnothing = \varepsilon$ follows from the base case $\neg\varnothing \setminus r = \varepsilon$ of `reSubtraction`.

Right identity. `e >>= pure = e`.
ret: $ret(e)$. ✓ *post*: $post(e) <> \varepsilon = post(e)$. ✓
Let $fe = \texttt{pure}(ret(e))$. Then $post(fe) = \varepsilon$ and

$$\widehat{future}(fe) = \texttt{future}(fe)\big(ret(fe)\big) = (\lambda\_.\, \neg\varnothing)\big(ret(e)\big) = \neg\varnothing.$$

$$\widehat{future}(e \gg\!= \texttt{pure}) = \big(\varepsilon \setminus \widehat{future}(e)\big) \wedge \neg\varnothing \;=\; \widehat{future}(e) \wedge \neg\varnothing \;=\; \widehat{future}(e).\ \checkmark$$

Associativity. For $(e \gg\!= f) \gg\!= g = e \gg\!= (\lambda x.\, f\, x \gg\!= g)$, the *ret* and *post* components follow from associativity of $\diamond$. For the future, let $r_1 = ret(e)$, $r_2 = ret(f\, r_1)$, $r_3 = ret(g\, r_2)$. Both sides reduce to

$$\big(post(g\, r_2) \setminus \big(post(f\, r_1) \setminus \widehat{future}(e)\big)\big) \;\wedge\; \widehat{future}(f\, r_1) \;\wedge\; \widehat{future}(g\, r_2)$$

by repeated application of Equation (3) and associativity of $\wedge$.

Precondition. The `pre` field propagates constraints via $\wedge$. When $post(e)$ fully covers $pre(e_2)$, Equation (4) gives $pre(e \gg\!= f) = pre(e) \wedge \varepsilon$, which reduces to ε when $pre(e) = \neg\varnothing$ or $\varnothing$ when $pre(e)$ is non-nullable, flagging contradictions immediately. The residual $post(e) \setminus pre(e_2)$ records precisely which precondition obligations remain unmet.

10 Related Work

Pre/post-condition contracts in Haskell. Liquid Haskell [Vazou et al. 2014] expresses pre/post-conditions as refinement types verified by an SMT solver. Findler and Felleisen [Findler and Felleisen 2002] establish a runtime contract discipline for higher-order functions. QuickCheck [Claessen and Hughes 2000] supports conditional properties via `==>`. Swierstra and Baanen [Swierstra and Baanen 2019] embed Hoare triples as an indexed monad, propagating pre/post annotations through monadic bind analogously to `Pledge`. Pledge complements these by adding a *temporal* dimension through the future condition, without requiring type-system extensions.

Future conditions. Song et al. propose future conditions as a first-class extension to pre/post-condition contracts [Song et al. 2025], formalising the notion that an operation may discharge temporal obligations onto its continuation. Pledge realises this idea as a Haskell library, contributing a monadic interface, derivative-based propagation, and four concrete `Composable` instances that the formal system leaves unaddressed.

Parameterised and graded monads. Atkey [**?**] introduces parameterised monads $T(S_1, S_2, A)$ for Hoare-logic-style pre/post state tracking. Katsumata [Katsumata 2014] generalises this to graded monads over a monoid; Orchard and Wu [Orchard and Wu 2014] show effect systems and session types are instances. Pledge is related but orthogonal: rather than indexing by effect *type* at the type level, we carry an explicit obligation as `future :: a → eff` at the value level, preserving compatibility with the standard `Monad` class, **`do`**-notation, and the `mtl` stack.

Temporal property verification and repair. ProveNFix [Song et al. 2024] encodes temporal properties as regular expressions to guide automated program repair, verifying that synthesised patches satisfy the specification. Pledge is complementary in orientation: rather than repairing programs after the fact, it carries temporal obligations as first-class annotations so that violations are identified statically, prior to any execution.

Resource management in Haskell. `bracket` lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. `ResourceT` [**?**] threads a finaliser queue through every bind, discharging cleanup when the block exits. `Acquire` and `Managed` generalise this to an applicative/monadic

interface. All three handle the *cleanup* half of a resource obligation but cannot express *protocol* or *quantitative* obligations, which require explicit future-condition propagation across bind steps.

Linear types. Linear Haskell [?] introduces multiplicity annotations enforced by GHC since version 9.0; `linear-base` [?] provides a `RIO` monad with linear file handles, statically guaranteeing every opened handle is consumed exactly once. Linear types enforce *structural* obligations (use exactly once), whereas future conditions enforce *temporal* obligations (use in a specified order, or with a specified probability). A hybrid combining linear types for uniqueness with Pledge for temporal ordering could achieve stronger guarantees than either alone.

Effect systems and algebraic effect handlers. Algebraic effect handlers [Pretnar 2015] decompose effectful computations into an operation set and a separate handler; resource lifecycle obligations are typically encoded in the handler's state machine. Polysemy's `Scoped` effect attaches resource creation and cleanup to a lexical scope threaded through the effect row. Pledge takes the complementary route: obligations are values in the annotation fields, composable across any handler.

Session types. Session types [Gay and Hole 2005; Honda 1993] enforce communication protocols at the type level. The `sessiontypes` library [?] realises this via an indexed monad `STTerm m s s' a`; the type checker rejects programs that violate the protocol order. Pledge is more lightweight: it operates within the standard `Monad` class, admits quantitative and heap-ownership obligations that session types do not, and is applicable beyond message-passing.

Typestate. Typestate systems [Bierhoff and Aldrich 2007; Strom and Yemini 1986] encode resource lifecycle protocols in the type system. Brady's `ST` monad encoding in Idris [?] demonstrates full compile-time enforcement in a dependently typed language. Pledge achieves comparable expressiveness at the library level using standard Haskell type classes, at the cost of moving checking to annotation-inspection rather than type-inference time.

Runtime monitoring. Runtime verification [Leucker and Schallhart 2009] checks temporal properties after execution. Pledge takes a *static* approach: future conditions are propagated before any execution, and the LTL_f embedding eliminates automaton construction.

Brzozowski derivatives. Derivatives of regular expressions [Brzozowski 1964] have found application in parsing [Might et al. 2011; Owens et al. 2009] and model checking. Equation (1) extends this to complement via $\partial_a(\neg r) = \neg(\partial_a r)$; our subtraction operator is a novel application to future-condition propagation in a monadic setting.

11 Discussion and Future Work

Decidability. Equality of normalised regular expressions is decidable, so the fixed-point computation in `subtraction` terminates whenever the derivative sequence is finite, which holds for the regular languages used in our examples. Extending to context-free or ω-regular specifications remains future work.

Precondition monad laws. The `pre` field satisfies the Hoare-rule property rather than strict monad-law equality (Section 9). A natural refinement is to make `pre` a separate, non-monadic annotation computed by a static analysis pass over the `Pledge` term, separating the behavioural monad from the precondition contract checker.

Integration with effect handlers. Lifting Pledge into an effect-handler framework (`effectful` or `polysemy`) would allow temporal specifications to compose alongside algebraic effect rows. A lightweight *shadow* approach requires no deep integration: keep the `Pledge RE` computation as a

pure specification evaluated statically, while a separate `Eff es` computation carries out the real effects, with assertions linking the two at the top level:

```
main = do
  assert (normalize (pre        spec) == top) "precondition␣violated"
  assert (normalize (evalFuture spec) == top) "future␣obligation␣unmet"
  runEff impl
```

Static checking is entirely pure; no changes to the effect stack are required.

Closed-world alphabet assumption. `firstWith` uses $\Sigma_0 = atoms(r_1) \cup atoms(r_2)$ as its working alphabet. This is sound under the *closed-world assumption* that every relevant event is mentioned in the two operands. A fully open-world implementation would accept an explicit Σ declaration, enabling complement over an unrestricted domain.

Connection to indexed and graded monads. Because `future` $:: a \to \mathit{eff}$ indexes the obligation by the return value, Pledge echoes the indexed/graded-monad literature where types are indexed by computational effects; here the index is a return value instead of an effect row.

Solver-backed discharge of residual conditions. Residual annotations are currently surfaced as symbolic expressions. A natural extension would connect them to an automatic solver: RE residuals checked by NFA/DFA construction or a BDD library; SL predicates with Presburger guards discharged by Z3 or CVC5, reporting concrete counterexample traces or unsatisfiable heaps.

Parallel composition. `Pledge` is inherently sequential; concurrent programs require a parallel combinator $e_1 \parallel e_2$. In the RE instance this corresponds to the shuffle product $r_1 \sqcup\!\sqcup r_2$; the SL instance admits a parallel connective via the concurrent separation logic frame rule [O'Hearn 2007]. Extending `Composable` with a `parallel` operation is a natural next step.

Quantitative future conditions. A natural further direction is to connect WRE to probabilistic temporal logics (PCTL) or quantitative Kleene algebras; in the SL instance, a quantitative analogue could express amortised resource bounds or expected heap usage.

12 Conclusion

We have presented *Pledge*, a Haskell library that augments effectful computations with *future conditions*: annotations that constrain what the remainder of a program must accomplish. Four design decisions underpin the approach.

First, the `Composable` typeclass abstracts the specification algebra into five operations: concatenation, conjunction, subtraction, and their respective identity elements. It admits four independent instances that all share the same monadic bind formula. This uniformity demonstrates that trace-based, arithmetic, quantitative, and heap-ownership obligations are not merely analogous, but structurally identical at the algebraic level.

Second, the RE instance incorporates complement as a first-class constructor, exploiting the single law $\partial_a(\neg r) = \neg(\partial_a r)$ to support globally assertions and a direct embedding of LTL_f, without any automaton construction.

Third, the GRE instance pairs each RE with a Presburger arithmetic predicate, so that a single annotation simultaneously enforces a protocol ordering and a numeric bound; the `normalizePPred` procedure eagerly introduces the canonical **`PFalse`** constructor, identifying syntactically unsatisfiable constraints before any solver call is issued.

Fourth, the WRE instance generalises Boolean membership to a semiring weight, unifying probabilistic future conditions under the probability semiring and min-cost scheduling obligations

under the tropical semiring within a single framework. The SL instance further demonstrates the breadth of the `Composable` interface by instantiating separating conjunction as sequential composition and magic wand as subtraction, with `Pure PPred` inside `Conj` making value-dependent preconditions such as "balance ≥ amount" expressible as first-class heap predicates.

The resulting system is lightweight (no language extensions beyond standard Haskell type classes), composable via a standard `Monad` instance, and diagnostically precise across all four dimensions: a residual `future` identifies unmet trace obligations; a **`PFalse`** in `pre` flags an arithmetic violation; a semiring weight of `szero` signals an unfulfilled quantitative obligation; and a leaked `Cell` in a residual `post` exposes a heap-ownership error.

References

Kevin Bierhoff and Jonathan Aldrich. 2007. Modular Typestate Checking of Aliased Objects. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA)*. 301–320. doi:10.1145/1297027.1297050

Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (1964), 481–494. doi:10.1145/321239.321249

Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 268–279. doi:10.1145/351240.351266

Robert Bruce Findler and Matthias Felleisen. 2002. Contracts for Higher-Order Functions. In *Proceedings of the 7th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 48–59. doi:10.1145/581478.581484

Simon J. Gay and Malcolm Hole. 2005. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42, 2-3 (2005), 191–225. doi:10.1007/s00236-005-0177-z

Kohei Honda. 1993. Types for Dyadic Interaction. In *Proceedings of the 4th International Conference on Concurrency Theory (CONCUR) (Lecture Notes in Computer Science, Vol. 715)*. 509–523. doi:10.1007/3-540-57208-2_35

Shin-ya Katsumata. 2014. Parametric Effect Monads and Semantics of Effect Systems. *ACM SIGPLAN Notices* 49, 1 (2014), 633–645. doi:10.1145/2578855.2535846

Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004

Matthew Might, David Darais, and Daniel Spiewak. 2011. Parsing with Derivatives: A Functional Pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 189–195. doi:10.1145/2034773.2034801

Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. In *Theoretical Computer Science*, Vol. 375. 271–307. doi:10.1016/j.tcs.2006.12.035

Dominic Orchard and Nicolas Wu. 2014. Effects as Sessions, Sessions as Effects. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 568–580. doi:10.1145/2535838.2535846

Scott Owens, John Reppy, and Aaron Turon. 2009. Regular-expression Derivatives Re-examined. In *Journal of Functional Programming*, Vol. 19. 173–190. doi:10.1017/S0956796808007090

Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. In *Electronic Notes in Theoretical Computer Science*, Vol. 319. 19–35. doi:10.1016/j.entcs.2015.12.003

Yahui Song, Darius Foo, and Wei-Ngan Chin. 2025. Specifying and Verifying Future Conditions. In *Static Analysis — 32nd International Symposium, SAS 2025 (Lecture Notes in Computer Science, Vol. 16100)*. Springer, 90–112. doi:10.1007/978-3-032-07106-4_5

Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin, and Abhik Roychoudhury. 2024. ProveNFix: Temporal Property-Guided Program Repair. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 11 (2024), 23 pages. doi:10.1145/3643737

Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* 12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929

Wouter Swierstra and Tim Baanen. 2019. A Predicate Transformer Semantics for Effects (Functional Pearl). *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 103 (2019), 26 pages. doi:10.1145/3341707

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 269–282. doi:10.1145/2628136.2628161